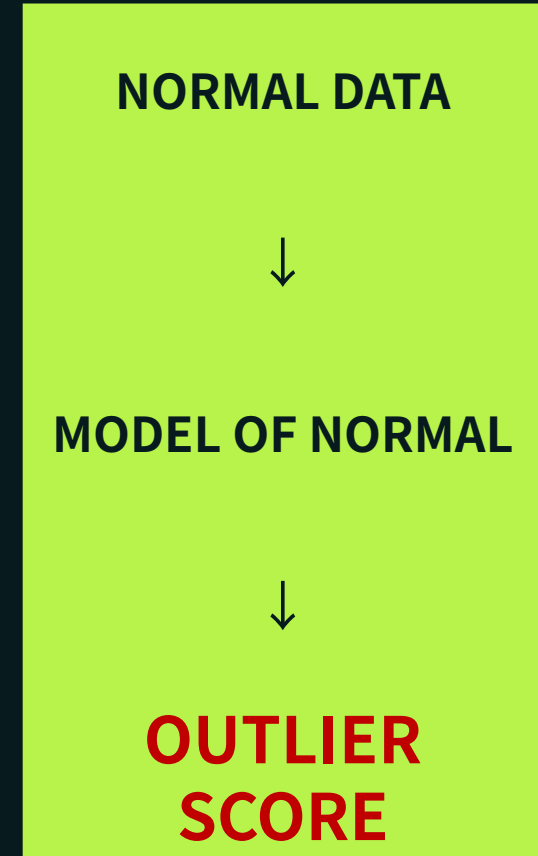


Outlier Detection

Unsupervised Methods

Finding the odd ones out when nobody has labelled them



Four model families learn what normal looks like

Every method here is unsupervised — no outlier labels are needed

01	————	One-Class SVM & SVDD	a boundary around normal data hyperplane · smallest hypersphere
02	————	Isolation Forest	outliers are easy to isolate random splits · path length · anomaly score
03	————	Local Outlier Factor	density relative to neighbours reachability distance · lrd · LOF ratio
04	————	Autoencoders	outliers reconstruct badly normal-only training · error · LSTM

Start with domain sense, then move to multivariate models

Three levels of outlier detection, from simplest to most flexible

LEVEL 1

Domain-based EDA

- Plots and business rules
- Know what is impossible

LEVEL 2

Univariate rules

- Tukey fences (IQR)
- z-score thresholds

LEVEL 3 · THIS DECK

Multivariate models

- MCD · One-Class SVM
- Isolation Forest · LOF
- Autoencoders

One variable at a time misses points that are odd only in combination.

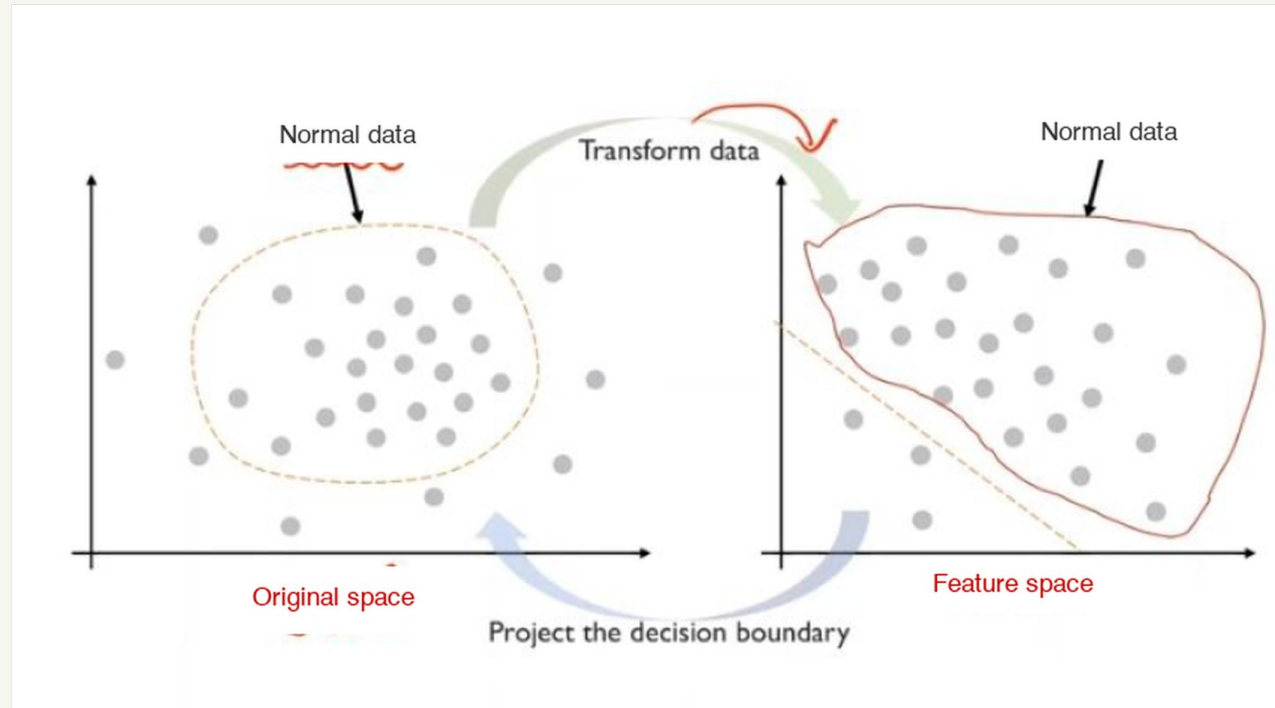
01

One-Class SVM & SVDD

Draw a boundary around the normal data, flag what falls outside

One-Class SVM pushes the normal data away from the origin

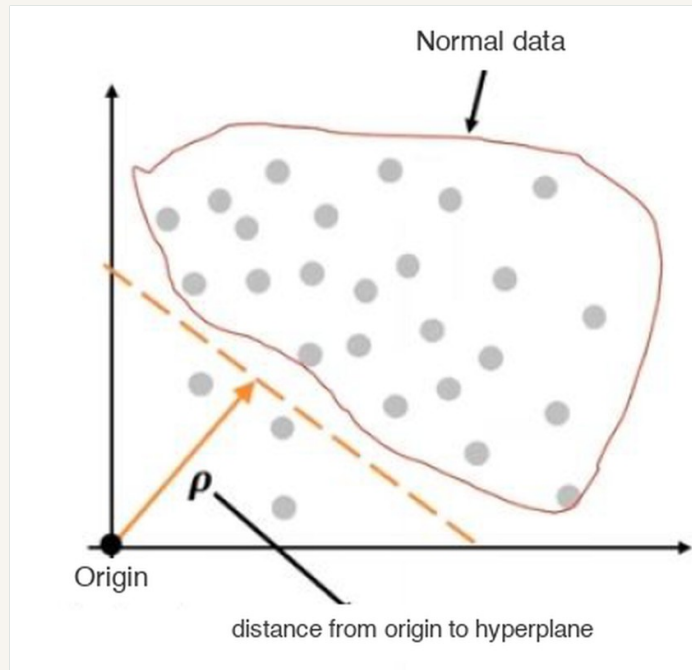
Map the data to a feature space, separate it from the origin, project the boundary back



※ An SVM trained on one class only: it learns the region of normal observations, not the outliers

The hyperplane sits as far from the origin as possible

ρ = distance from the origin to the hyperplane; v bounds the share of outliers



Objective

$$\min_{w, \xi, \rho} \frac{1}{2} \|w\|^2 + \frac{1}{vn} \sum_{i=1}^n \xi_i - \rho$$

Constraints

$$\text{s. t. } w \cdot \phi(x_i) \geq \rho - \xi_i, \quad \xi_i \geq 0$$

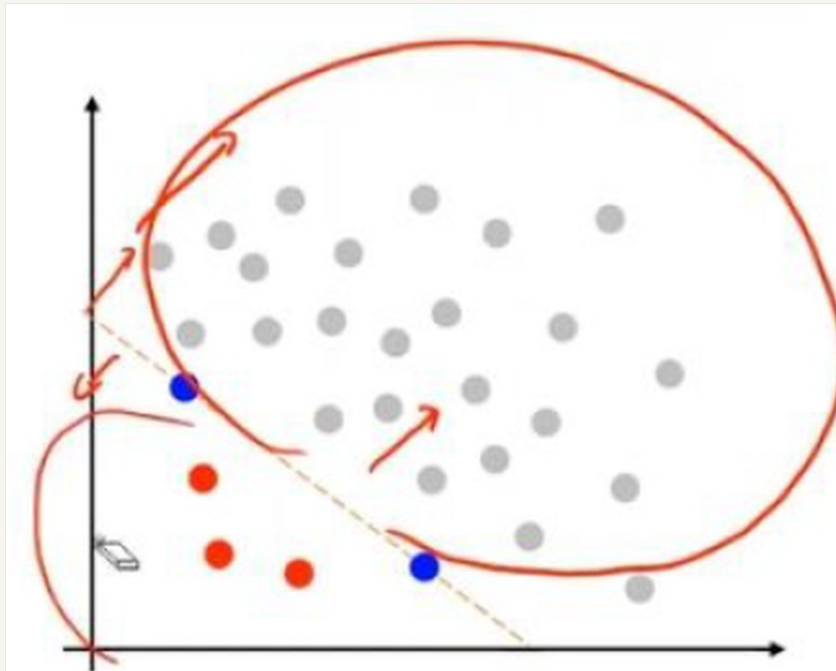
Decision: +1 normal · -1 outlier

$$f(x) = \text{sign}(w \cdot \phi(x) - \rho)$$

Maximise ρ while letting a few points (slack ξ) cross — those are the outliers.

The multiplier α tells where each observation sits

From the dual problem of the One-Class SVM



Dual constraints

$$\alpha_i = \frac{1}{\nu n} - \beta_i, \quad \sum_i \alpha_i = 1, \quad 0 \leq \alpha_i \leq \frac{1}{\nu n}$$

- $\alpha_i = 0$ inside the boundary — normal
- $0 < \alpha_i < \frac{1}{\nu n} \Rightarrow \xi_i = 0$ on the boundary — support vector
- $\alpha_i = \frac{1}{\nu n} \Rightarrow \xi_i > 0$ outside — support vector, outlier

Only the support vectors ($\alpha > 0$) shape the boundary.

In scikit-learn, predict returns -1 for an outlier

sklearn.svm.OneClassSVM — five one-dimensional points

READING THE OUTPUT

- 0 and 1 are flagged -1
- score_samples: lower = odder
- gamma sets the kernel width

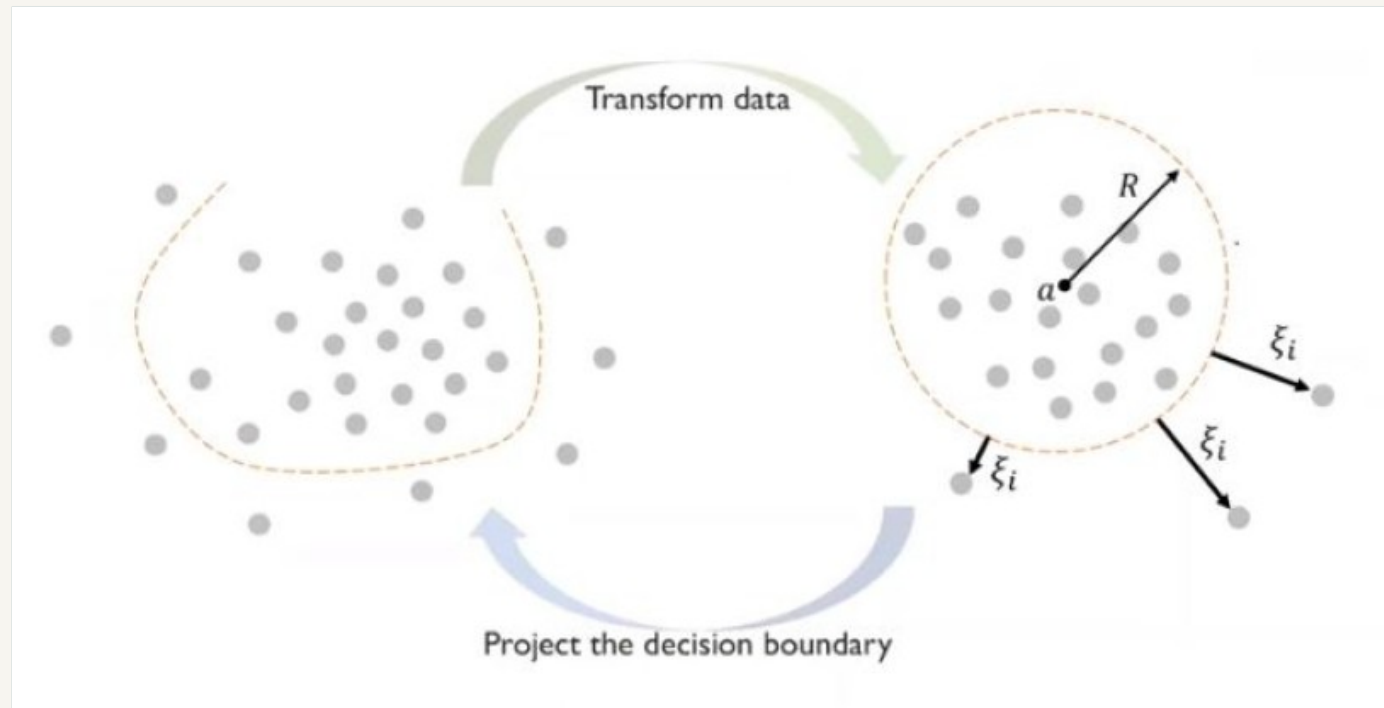
Fit on normal data, then score new data

Examples

```
>>> from sklearn.svm import OneClassSVM
>>> X = [[0], [0.44], [0.45], [0.46], [1]]
>>> clf = OneClassSVM(gamma='auto').fit(X)
>>> clf.predict(X)
array([-1,  1,  1,  1, -1])
>>> clf.score_samples(X)
array([1.7798..., 2.0547..., 2.0556..., 2.0561..., 1.7332...])
```


SVDD wraps the normal data in the smallest hypersphere

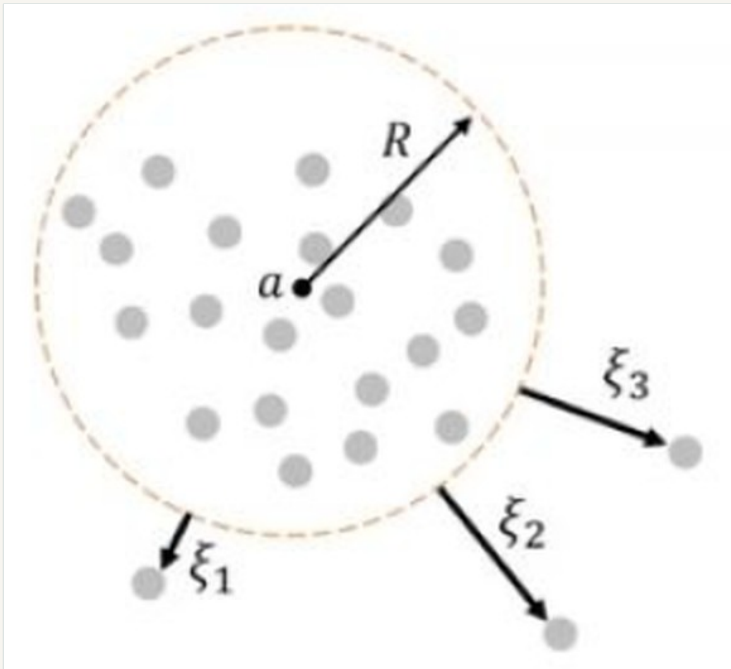
Support Vector Data Description — centre a , radius R , slack ξ for points outside



※ Same transform-then-project idea as the One-Class SVM; the boundary is a sphere instead of a hyperplane

SVDD trades a small radius against points left outside

ϕ = kernel mapping · C = regularisation · n = number of normal observations



Objective: shrink the sphere, penalise distances outside

$$\min_{R, a, \xi} R^2 + C \sum_{i=1}^n \xi_i$$

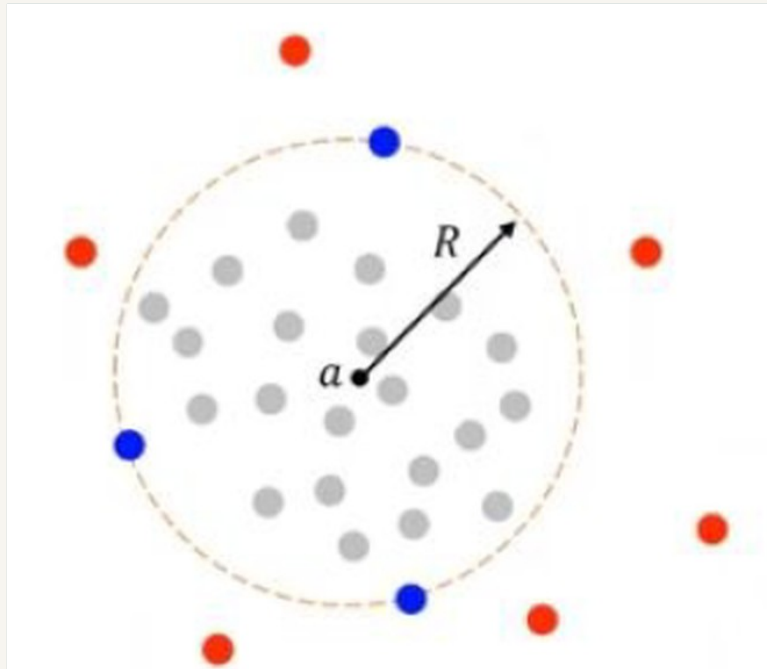
Constraints

$$\text{s. t. } \|\phi(x_i) - a\|^2 \leq R^2 + \xi_i, \quad \xi_i \geq 0$$

Large C keeps almost every point inside; small C allows a tighter sphere and more outliers.

As in the One-Class SVM, α places each observation

From the dual problem of SVDD



Dual constraints

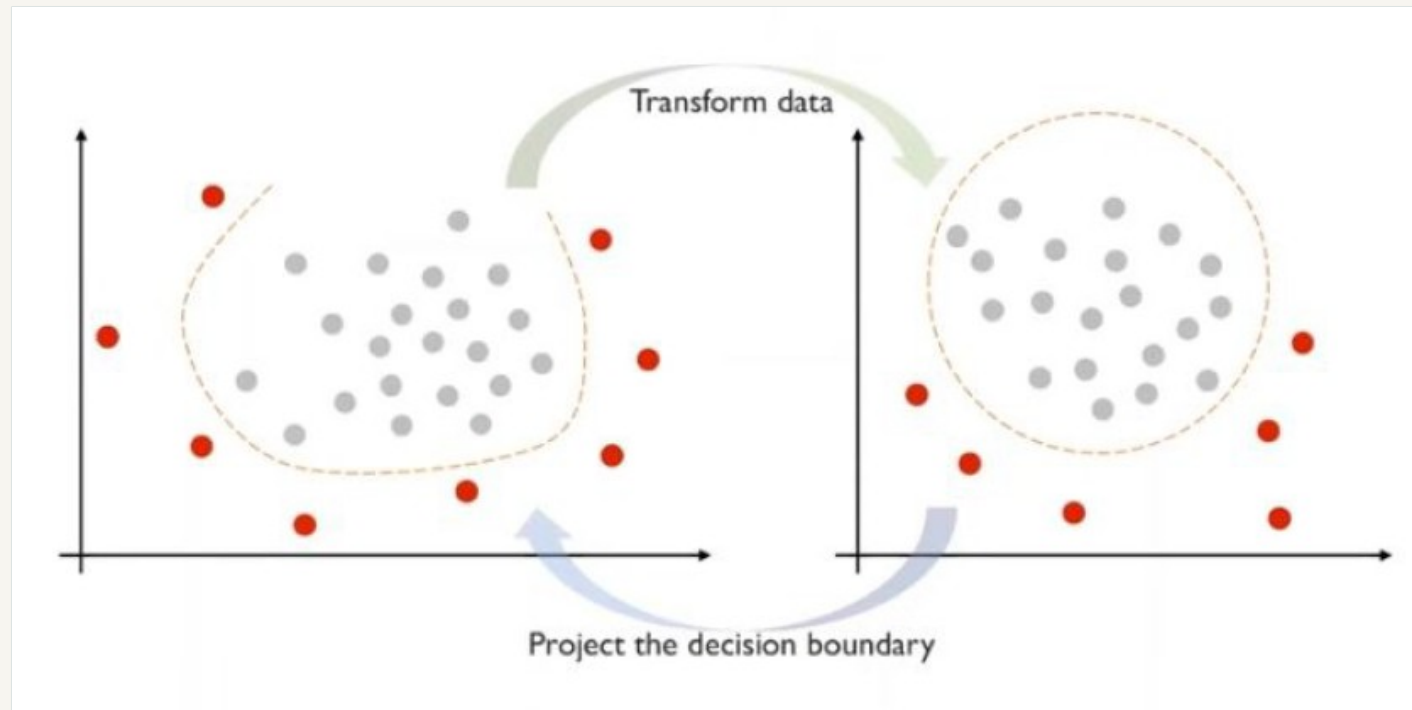
$$C - \alpha_i - \beta_i = 0, \quad \beta_i \xi_i = 0, \quad 0 \leq \alpha_i \leq C$$

- $\alpha_i = 0$ inside the sphere — normal
- $0 < \alpha_i < C \Rightarrow \xi_i = 0$ on the sphere — support vector
- $\alpha_i = C \Rightarrow \xi_i > 0$ outside — support vector, outlier

Points outside the sphere are the outliers.

Observations outside the sphere are judged outliers

Find the hypersphere around normal data in feature space, then project its boundary back



※ Implementation walk-through: zephyrus1111.tistory.com/471

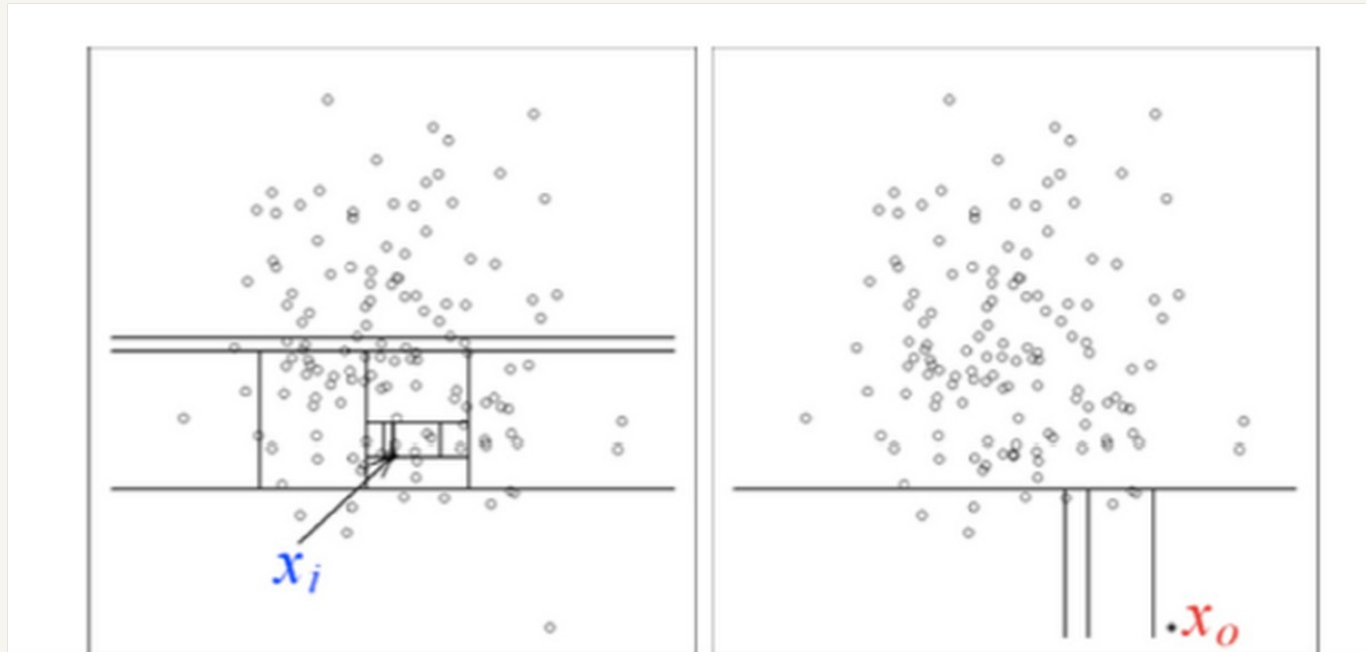
02

Isolation Forest

Outliers are few and different, so they are easy to isolate

A normal point needs many cuts; an outlier needs few

Each iTree splits at random to isolate observations — no labels needed



The point in the crowd takes many splits; the lone point takes only a few.

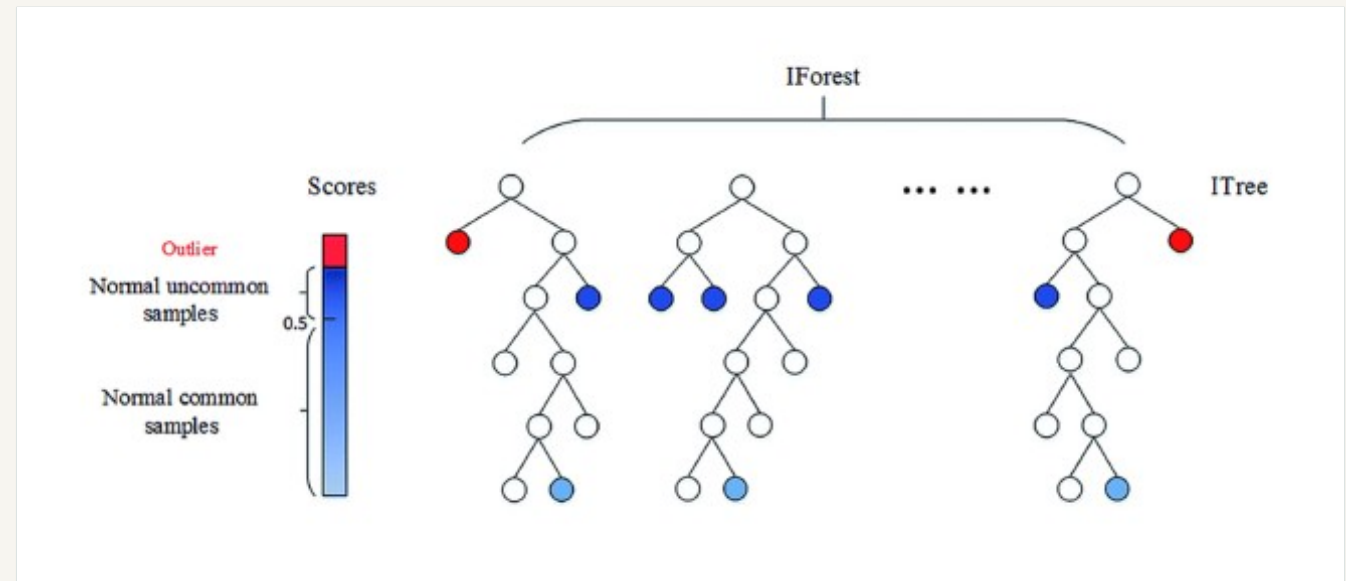
Random splits on random features build each tree

Repeat until every point is isolated or a depth limit is reached

HOW ONE ITREE GROWS

- Pick a feature at random
- Cut uniformly in [min, max]
- Record each path length

Short path = likely outlier



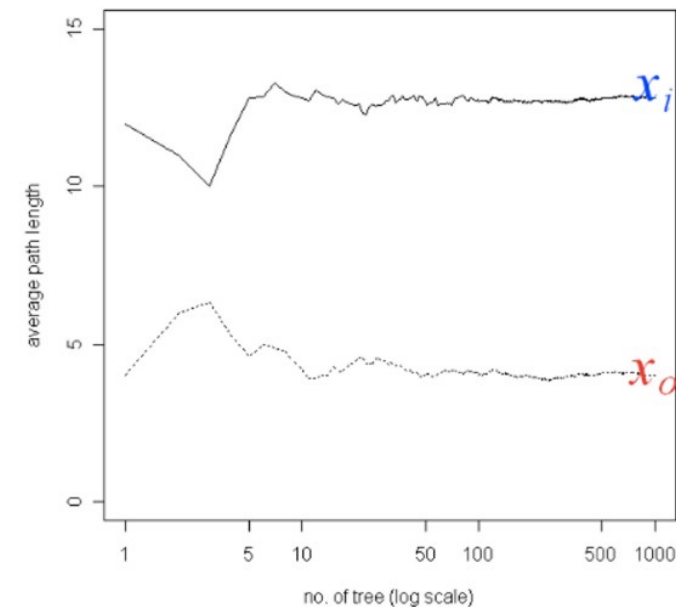
Averaged over many trees, the path lengths settle

Average path length of the crowded and the lone point as the number of trees grows (log scale)

WHAT THE CHART SHOWS

- Crowded point: about 12–13
- Lone point: about 4
- Stable after ~100 trees

The forest separates the outlier robustly



(c) Average path lengths converge

The anomaly score turns path length into a number near 0.5 or 1

$h(x)$ = path length in one tree · $c(n)$ = average path length for n points, used to normalise

Anomaly score

$$s(x, n) = 2^{-E[h(x)] / c(n)}$$

Normaliser

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(i) \approx \ln i + 0.5772$$

READING THE SCORE

$s \rightarrow 1$: outlier

$s \approx 0.5$: normal

$s < 0.5$: deep in the crowd

If $E[h(x)]$ equals $c(n)$, the score is exactly $2^{-1} = 0.5$.

In scikit-learn, IsolationForest flags the far value -1

sklearn.ensemble.IsolationForest — fit on four values, predict three new ones

READING THE OUTPUT

- 0.1 and 0 → +1 (normal)
- 90 → -1 (outlier)
- No labels used in fit

-1 = outlier, +1 = inlier

Examples

```
>>> from sklearn.ensemble import IsolationForest
>>> X = [[-1.1], [0.3], [0.5], [100]]
>>> clf = IsolationForest(random_state=0).fit(X)
>>> clf.predict([[0.1], [0], [90]])
array([ 1,  1, -1])
```

03

Local Outlier Factor

Compare a point's density with the density of its neighbours

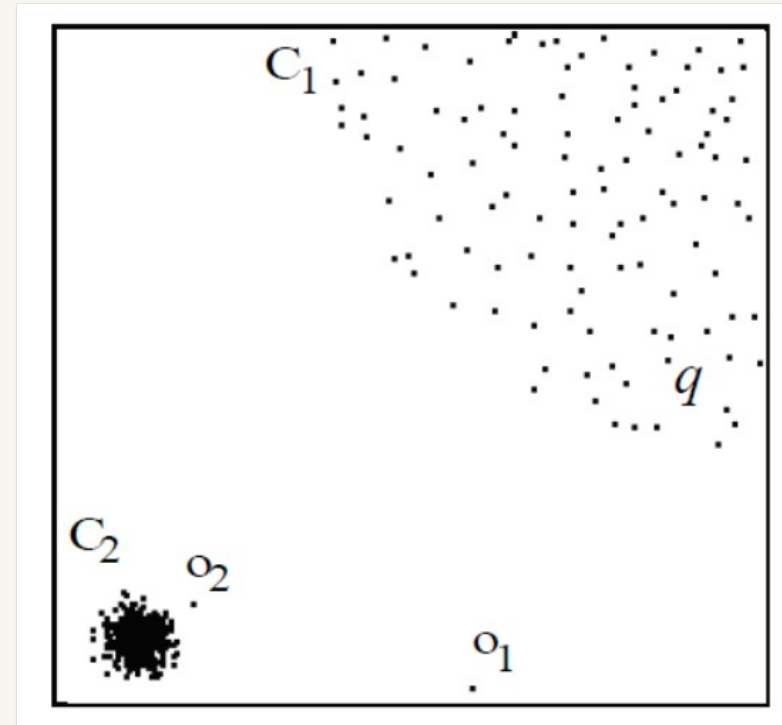
Is O2 an outlier relative to cluster C2?

A sparse cluster C1, a dense cluster C2, and two candidate outliers

THE PROBLEM

- O1 is far from everything
- O2 is close in absolute terms
- ...but far for dense C2

A global distance cutoff misses O2



Local reachability density is the inverse of average reach distance

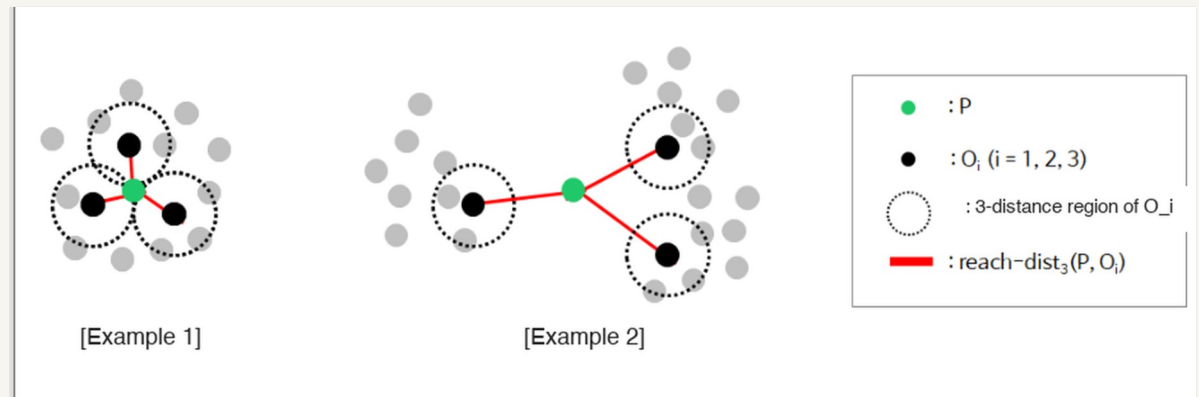
$\text{lrd}_k(p)$: how tightly packed the k nearest neighbours of p are — here $k = 3$

Reachability distance

$$\text{reach_dist}_k(p, o) = \max \{ k_dist(o), d(p, o) \}$$

Local reachability density

$$\text{lrd}_k(p) = \frac{|N_k(p)|}{\sum_{o \in N_k(p)} \text{reach_dist}_k(p, o)}$$



Long reach distances to the neighbours mean low density (Example 2).

LOF compares the neighbours' density with the point's own

The average ratio of $\text{lrd}(o)$ to $\text{lrd}(p)$ over the k neighbours o of p

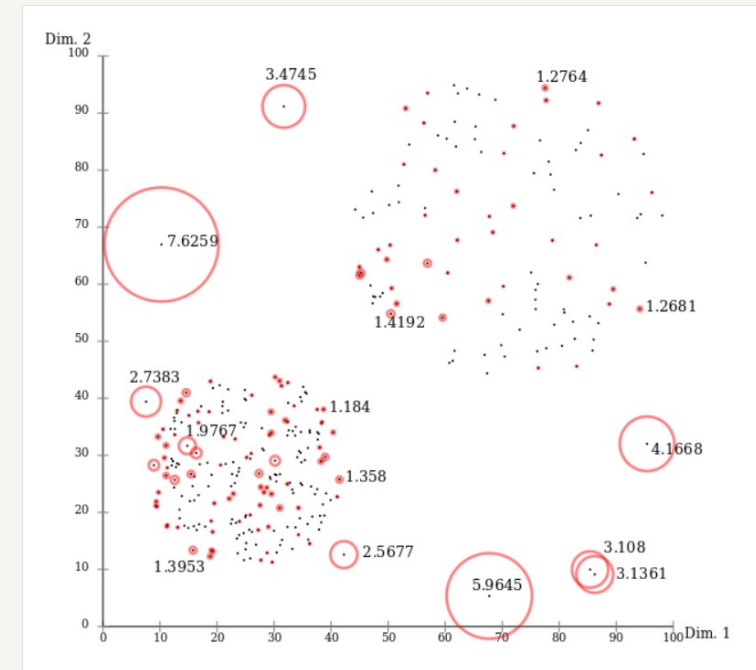
Local Outlier Factor

$$\text{LOF}_k(p) = \frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)}$$

LOF < 1 denser than its neighbours

LOF $\approx$ 1 similar to its neighbours

LOF $\gg$ 1 sparser — an outlier



Isolated points score about 2.5 to 7.6; clustered points stay near 1.

In scikit-learn, fit_predict labels the far point -1

sklearn.neighbors.LocalOutlierFactor with n_neighbors = 2

READING THE OUTPUT

- 101.1 is flagged -1
- negative_outlier_factor_ = -LOF
- -73.4 vs about -1

More negative = more outlying

Examples

```
>>> import numpy as np
>>> from sklearn.neighbors import LocalOutlierFactor
>>> X = [[-1.1], [0.2], [101.1], [0.3]]
>>> clf = LocalOutlierFactor(n_neighbors=2)
>>> clf.fit_predict(X)
array([ 1,  1, -1,  1])
>>> clf.negative_outlier_factor_
array([-0.9821..., -1.0370..., -73.3697..., -0.9821...])
```

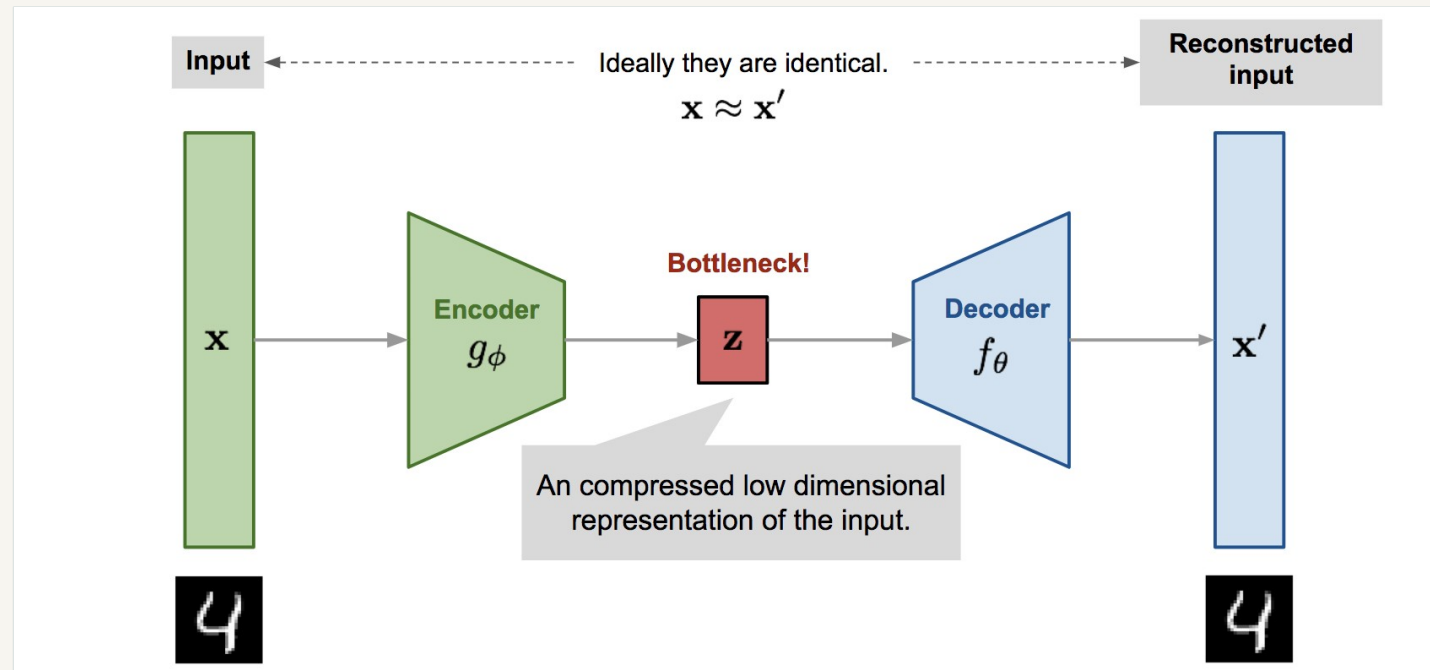
04

Autoencoders

A network trained on normal data reconstructs outliers badly

An autoencoder trained on normal data cannot rebuild an outlier

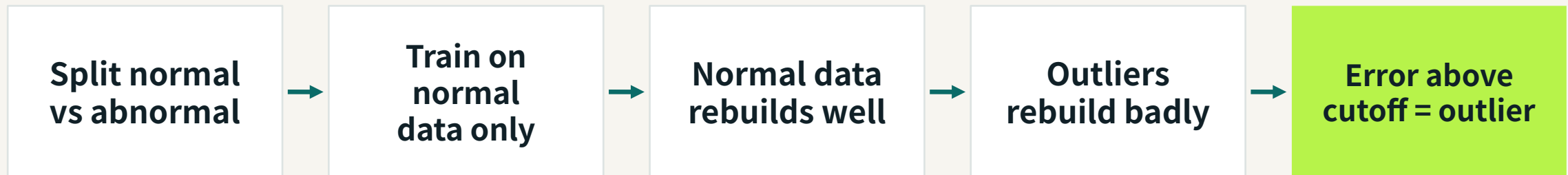
Encoder g_ϕ compresses x to a bottleneck z ; decoder f_θ rebuilds $x' \approx x$



Ideally $x' \approx x$ — the bottleneck keeps only the main features of normal data.

High reconstruction error marks an outlier

Five steps of autoencoder-based outlier detection



Reconstruction error and cutoff τ

$$e(x) = \|x - f_{\theta}(g_{\phi}(x))\|^2 > \tau \Rightarrow \text{outlier}$$

The network never saw outliers in training, so it cannot rebuild them.

Label as outliers the data whose reconstruction error exceeds τ .

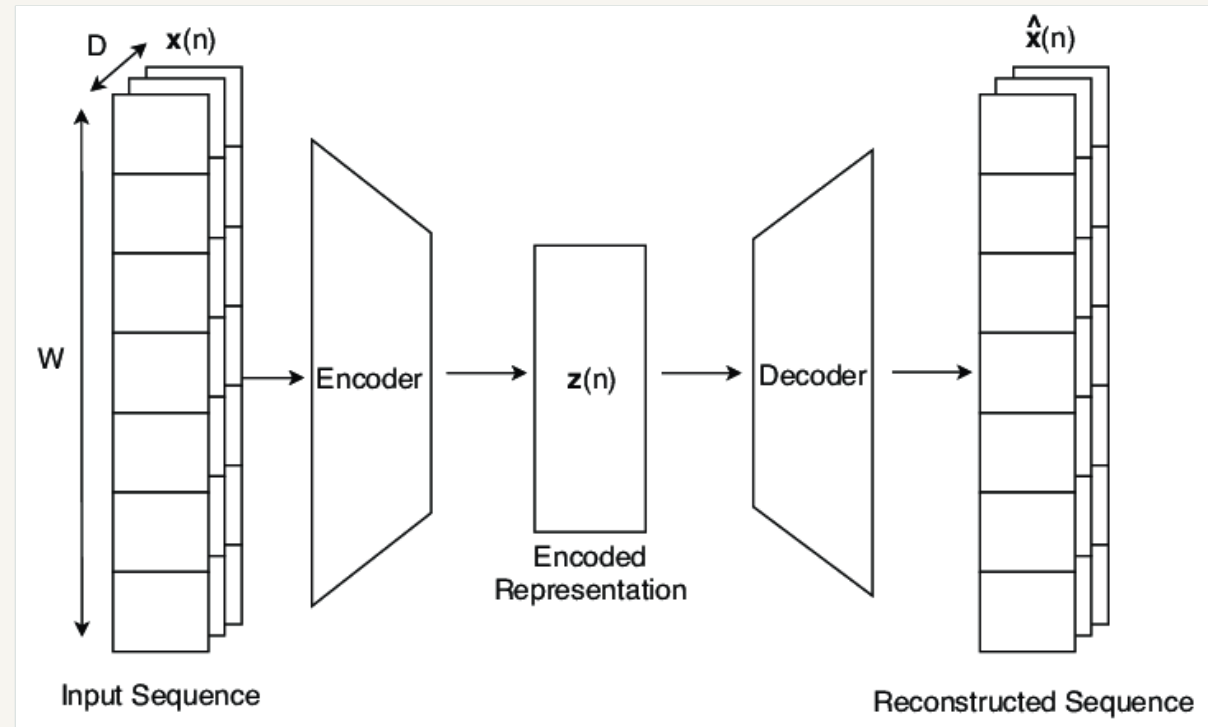
An LSTM autoencoder applies the same idea to sequences

Encode a window of time steps and features, rebuild it, score the error

TIME-SERIES OUTLIERS

- LSTM cells keep memory
- Error per window
- Spikes mark anomalies

Code: [cran2367](#) on GitHub



Each family defines 'normal' in a different way

Pick the one whose idea of normal matches your data

OCSVM · SVDD

A boundary

tune ν or C

ISOLATION
FOREST

Easy to isolate

score near 1

LOF

Locally sparse

$\text{LOF} \gg 1$

AUTOENCODER

Hard to rebuild

error above τ

No labels needed — but always check the flagged points with domain sense.

LEARN WHAT NORMAL LOOKS LIKE.

SCORE HOW FAR EACH POINT STRAYS.

FLAG THE ODD ONES OUT.

**Boundary, isolation, density,
reconstruction.**